Molarium: When Molecular Design Software Becomes Generated Infrastructure

A perspective on scientific contracts, validation, and the changing software layer in molecular design

Rafal P. Wiewiora Woody Sherman

PsiThera, Watertown, Massachusetts, USA

Perspective draft – August 2026

Abstract

Computational molecular design has traditionally bundled several distinct forms of value into the same software product: scientific models, numerical implementations, graphical interfaces, workflow orchestration, and the specialist knowledge required to operate them. Coding agents are beginning to separate these layers. Interfaces, analyses, integrations, workflow logic, and even bounded pieces of scientific numerical software can increasingly be constructed while the scientific question is still being formulated.

Molarium provides a concrete example of this transition. It is a local-first molecular workbench that runs primarily in a web browser and combines molecular visualization and editing, protein preparation, conformer exploration, molecular mechanics, molecular dynamics, machine-learned potentials, and protein-structure prediction. Its significance is not simply the breadth of those functions. Molarium was developed through a tight interaction between a drug-discovery scientist and a coding agent in which scientific objections – an implausible molecular geometry, an unexpected energy, an unsupported method, an ambiguous computational backend – became implementation changes, reference tests, and explicit limitations. The resulting software therefore provides a useful case study in what can now be generated and what still must be preserved.

The distinction is fundamental. Equations, interfaces, kernels, and workflow plumbing can often be reconstructed from sufficiently explicit specifications. Force-field parameterization, trained model weights, experimental evidence, applicability domains, and scientific judgment cannot. Molarium consequently treats each computational route as a scientific contract: the molecular state, model identity, parameters, numerical assumptions, scope, provenance, and validation evidence remain explicit even when the surrounding interface is fluid.

We argue that this pattern foreshadows a broader change in computational drug discovery. As implementation becomes less scarce, differentiation moves toward better models of molecular reality, better validation, proprietary experimental and organizational knowledge, and the scientific judgment required to decide when a result should be trusted. Molarium is therefore both a molecular-design workbench and an example of a more general idea: scientific software can increasingly be generated around the problem, but scientific truth cannot.

Keywords: agentic software development; computational chemistry; molecular design; WebGPU; scientific validation; discovery intelligence

1 From software products to generated scientific infrastructure

Computational molecular design has always required more software than its scientific algorithms alone would suggest. A docking calculation needs structures, protonation states, file conversion, parameter choices, visualization, job control, and analysis. Molecular dynamics adds system preparation, topology construction, force fields, solvation, constraints, integration, trajectory handling, and quality control. Free-energy methods introduce perturbation networks, sampling strategies, convergence diagnostics, and substantial execution infrastructure.

Similar layers surround cheminformatics, quantum chemistry, conformer generation, machine-learned potentials, property prediction, and protein-structure modeling.

Historically, much of this complexity was absorbed into scientific software products. That created enormous practical value. A well-designed application could make difficult calculations reproducible, hide irrelevant implementation detail, and allow scientists to focus on molecules rather than file formats and command lines. But it also coupled the scientific method to a relatively fixed interface and operating model. New questions frequently required new features; new features required engineering; and project-specific workflows competed with broader product priorities.

Coding agents change the latency of that loop.

A scientist can increasingly describe an analysis, interface, numerical test, or workflow in natural language; provide examples and reference values; inspect the resulting behavior; and ask the agent to modify the implementation immediately. Modern coding agents can inspect repositories, execute software, diagnose failures, write tests, and revise substantial portions of an application. Chemistry agents have similarly demonstrated that language models can coordinate established computational tools rather than merely describe them [1, 2]. The consequential change is not that software engineering disappears. It is that a growing portion of scientific implementation can be created during the scientific conversation itself.

Molarium emerged from exactly this process.

It began from a comparatively modest need: a better environment in which to look at and manipulate molecular structures. That request expanded through use. When a molecular edit produced chemically implausible geometry, the representation changed. When an energy appeared questionable, a reference calculation was introduced. When a browser calculation turned out not to be using the accelerator pathway that its interface appeared to imply, the architecture changed. When existing components did not provide the desired browser-native execution path, new numerical kernels were implemented and tested against independent references.

The result is not best understood as an exercise in generating a large number of features. Molarium instead illustrates a different relationship between a scientist and scientific software. The application is no longer only a finished product that the scientist learns to operate. It becomes an evolving computational environment that is shaped around the scientific questions encountered during use.

Molarium exposes a broader boundary: which parts of molecular-design software are becoming easy to reproduce, and which parts remain scientifically scarce?

2 Molarium as an adaptive molecular-design environment

Molarium is an MIT-licensed, local-first molecular viewer, builder, and simulation workbench. Most supported calculations execute on the user's device using WebAssembly or WebGPU. The workbench combines functions that would conventionally be distributed across molecular viewers, cheminformatics packages, simulation environments, notebooks, command-line tools, and remote services.

The architectural choice that matters most is not the browser itself. It is the preservation of a common scientific state.

A molecule is not passed casually from one loosely connected feature to another. Molarium maintains explicit chemical topology, coordinates, protonation state, user edits, preparation history, and provenance. Operations act on that state, and the resulting structures and trajectories remain connected to the methods that produced them. This allows the surrounding interface to change without silently changing the scientific object being studied.

Three task-specific views illustrate the design. **View** emphasizes the molecular system: protein, ligand, waters, ions, contacts, and selected residues. **Build** exposes chemical and geometrical interventions, including coupled bond-order, hydrogen, charge, and local-relaxation operations. **Inspect** emphasizes preparation findings, residue context, trajectories, and provenance. The controls differ because the scientific questions differ; the molecular state does not.

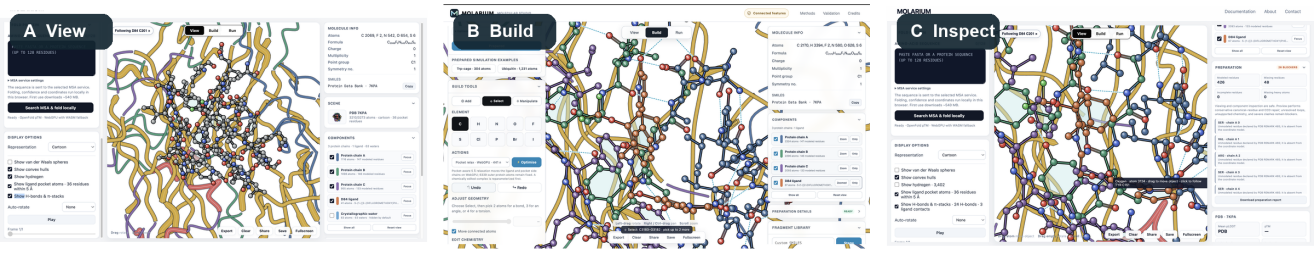

Figure 1. Molarium as an adaptive view over a persistent scientific state. Three views of the same 7KPA complex illustrate the principle. View emphasizes molecular context and interactions; Build exposes staged chemical and local structural edits; Inspect connects a selected structural region to preparation and trajectory information. The interface changes while topology, coordinates, provenance, and scientific context persist.

This separation becomes important when interfaces themselves are cheap to modify. If every generated view is permitted to redefine the molecule, select a different charge model, alter protonation, or reinterpret a trajectory, flexibility becomes a source of scientific ambiguity. Molarium therefore treats the molecular state and the identity of the computational method as invariants beneath a more fluid presentation layer.

The browser provides a useful implementation environment for this experiment. WebAssembly allows mature compiled scientific libraries to execute locally, while WebGPU exposes modern accelerators through a portable browser API [3, 4]. The same environment can provide molecular graphics, interactive editing, numerical work, analysis, and data provenance without requiring a conventional scientific software installation.

It is also a deliberately restrictive environment. There may be no Python interpreter, CUDA runtime, conventional filesystem, or unrestricted native dependency stack. WebGPU favors highly parallel single-precision arithmetic but does not reproduce every feature of a native GPU programming environment. These constraints forced scientific assumptions that might otherwise remain hidden inside a software stack to become explicit.

Molarium consequently uses several execution routes rather than presenting a single undifferentiated "energy" or "simulation" button. RDKit provides chemical perception, conformer embedding, and MMFF94 or UFF cleanup [5, 6, 7, 8]. Prepared OpenFF Sage systems can be evaluated using an OpenMM Reference implementation or browser-native WebGPU routes [9, 10]. Homogeneous replicas can be advanced through a STORMM-inspired browser engine [11]. ANI-2x provides a distinct machine-learned potential with its own elemental and molecular domain [12, 13]. A restricted OpenFold pathway provides protein-structure prediction under a separately defined set of assumptions [14].

These components coexist in one workbench, but they do not become one model.

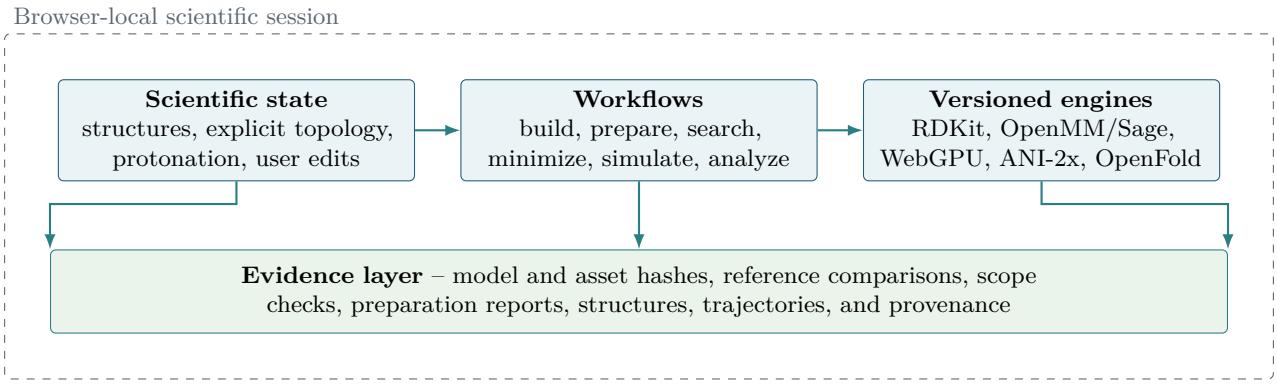

Figure 2. Molarium's local-first architecture. Named engines act on explicit molecular state; evidence and provenance remain attached. A new interface need not redefine the state, engine, or numerical scope.

3 Scientific contracts, not feature checklists

Scientific applications are often described in terms of capabilities: docking, dynamics, minimization, conformer search, prediction. Such descriptions are useful for comparing products, but they are insufficient for determining what a calculation means.

A molecular mechanics result is defined not merely by the fact that "molecular mechanics" was run. It depends on the topology, parameters, charge assignment, energy expression, treatment of nonbonded interactions, constraints, boundary conditions, precision, integrator, and numerical implementation. A learned potential is defined by its architecture, weights, supported chemistry, preprocessing, and training domain. Protein-structure prediction similarly depends on checkpoint identity, input processing, sequence information, recycle settings, templates, relaxation, and other protocol choices.

Molarium therefore treats a computational pathway as a **scientific contract**.

The contract includes the molecular state that entered the calculation; the identity and version of the model; parameter and model assets; numerical settings; declared scope; known exclusions; provenance; and the evidence used to determine whether the implementation behaves as claimed. The surrounding interface may be modified freely, but these quantities should not change invisibly.

This is particularly important in the Conformer Arena. Molarium can begin from symmetry-pruned ETKDG conformers, briefly clean them with MMFF94 or UFF, advance multiple conformers through a Sage/OBC2 replica workflow, and optionally evaluate supported structures with ANI-2x. These routes can be compared, but their energies are not interchangeable. Molarium can place a common declared score on common structures and show how different refinement strategies move through conformational space. It does not treat an absolute ANI-2x energy and a Sage energy as though subtraction between the two created a meaningful thermodynamic quantity.

The interface therefore exposes disagreement rather than forcing artificial consensus.

The same principle governs protein preparation. Supported residues and operations can be processed automatically, audited, and retained as provenance. Unsupported chemistry, severe structural pathologies, or assumptions beyond the defined preparation policy should stop or warn rather than being silently invented. An agentically generated workflow is most dangerous precisely when it fills gaps fluently enough that the user no longer notices where the scientific contract ended.

Molarium's local-first mode extends the contract to data handling. Connected functionality can explicitly contact external resources when required. Local Lab instead applies restrictive network controls and verifies the reviewed application assets. This should not be interpreted as an absolute privacy guarantee: software delivered by a hosted page necessarily depends on what that page serves, and the strongest boundary remains a reviewed local checkout on a controlled machine. The important design principle is that the network boundary is itself made inspectable and testable.

Fluid interfaces require stable scientific invariants.

If an agent can generate a new analysis view in minutes, that view must still expose which molecule, model, parameters, assumptions, and evidence produced the result. Otherwise the reduced cost of software generation merely increases the rate at which scientific ambiguity can be created.

4 The scientist-agent build loop

Molarium did not emerge from a single prompt, nor did the scientist provide a complete specification in advance. Development instead proceeded through repeated scientific challenges.

Early failures were often visual. A benzene geometry was wrong. A fragment acquired a spurious bond because spatial proximity was mistaken for chemical connectivity. A trifluoromethyl group arrived with implausible geometry. A bond-order edit triggered structural cleanup before the complete coupled chemical transformation had been applied.

These were not cosmetic defects. They exposed failures in molecular representation and editing semantics. The implementation consequently moved toward explicit topology, chemically staged edits, three-dimensional fragment coordinates, and local rather than indiscriminate global relaxation.

Other failures were numerical. An early evaluator prompted the question of whether the displayed energy actually corresponded to a recognized force field. That led to reference implementations and prepared OpenFF systems. A disagreement between CPU and GPU calculations revealed both a unit error and an architectural misunderstanding: an OpenMM Reference pathway compiled to WebAssembly was serving as a CPU numerical oracle rather than as the browser's WebGPU engine. The objection "is this actually running on the GPU?" therefore changed the implementation rather than merely the label.

Further questions drove further scientific infrastructure. Requesting implicit solvent led to an OBC2/ACE implementation [15, 16]. Asking whether constrained dynamics were genuinely supported led to SHAKE/RATTLE [17, 18]. A requested long simulation that unexpectedly terminated revealed that a development-time cap had leaked into product behavior.

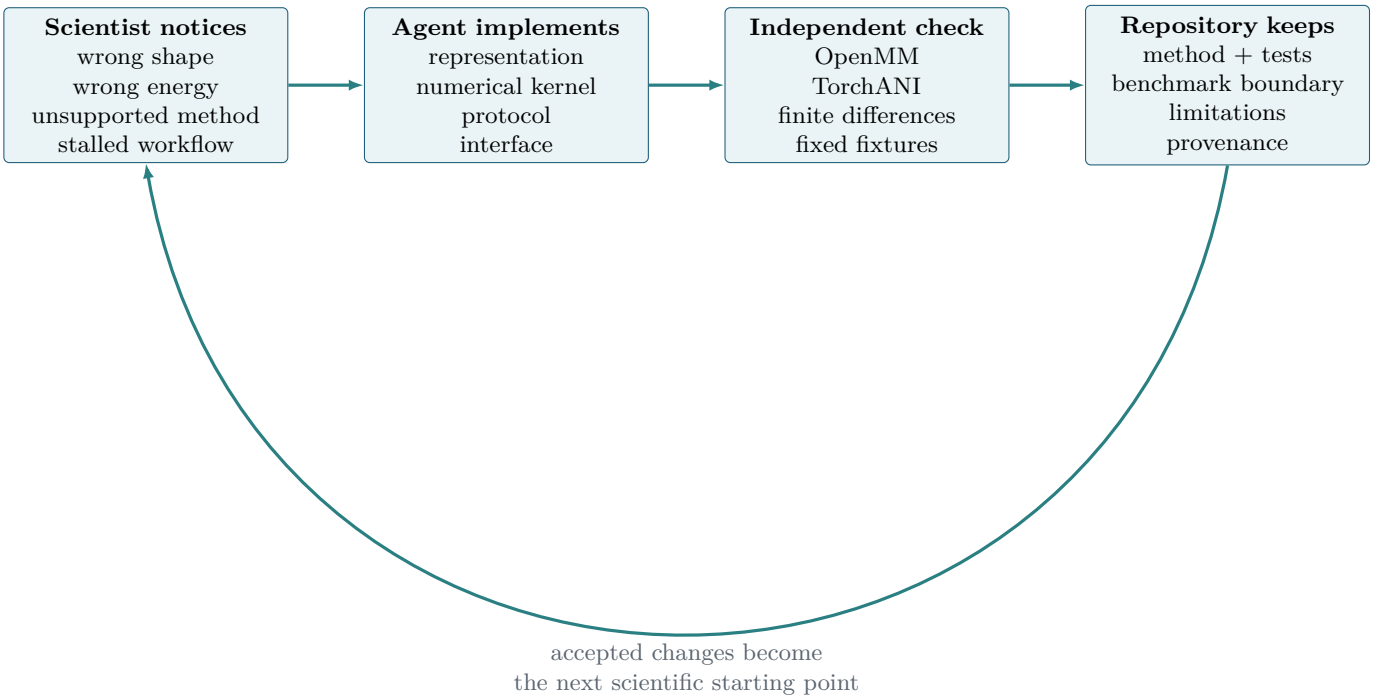

Figure 3. The Molarium scientific build loop. Scientific objections are converted into executable tests. The coding agent supplies implementation speed; independent references supply falsifiability; the repository retains the resulting method, benchmark boundary, limitations, and provenance.

The division of labor is important.

The coding agent can generate substantial software quickly. It can inspect a codebase, refactor an architecture, construct a WebGPU kernel, add tests, compare outputs, and repeat the loop without fatigue. But it does not independently determine which discrepancy is chemically important, which reference is genuinely independent, which approximation is acceptable for the intended use, or whether a numerical difference changes scientific interpretation.

The scientist provides those standards.

This is a different role from operating conventional scientific software. Expertise shifts away from remembering which menu contains a setting or how to prepare a particular input file and toward recognizing implausible results, defining meaningful controls, choosing the appropriate model, and deciding what evidence would change belief.

Agentic scientific software therefore does not eliminate expertise. It moves expertise closer to the scientific decision.

5 What Molarium generated – and what it preserved

The clearest lesson from Molarium is that "generated scientific software" does not mean that the underlying science was generated.

The project produced substantial new implementation: its persistent molecular canvas and topology-aware editing model; browser-specific preparation, conformer, trajectory, and analysis workflows; direct WebGPU evaluation of prepared Sage terms; OBC2/ACE calculations; homogeneous-replica dynamics with constraints; GPU-resident ANI descriptor and force calculations; local-mode controls; provenance views; and regression infrastructure.

At the same time, Molarium deliberately reused or preserved scientific assets whose value does not arise from the surrounding implementation.

Table 1. Generated infrastructure versus preserved scientific assets in Molarium.

Generated or substantially implemented in Molarium	Preserved or reused scientific assets
Molecular canvas, topology-aware builder, adaptive views	RDKit chemical perception, ETKDG, MMFF94, UFF
Browser preparation, conformer, trajectory, and inspection workflows	Established preparation rules and reference chemistry
Browser WebGPU evaluation of prepared Sage terms	OpenFF functional forms and parameter sets
Homogeneous-replica WebGPU mechanics and dynamics	STORMM concepts and independent upstream behavior
Browser ANI-2x descriptors and analytic force contraction	ANI-2x architecture and trained weights
Local Lab controls, provenance, regression fixtures	Upstream model evidence and external validation
Restricted browser OpenFold integration	OpenFold checkpoint weights and accumulated training evidence

This division is more consequential than the amount of code involved.

A published force-field equation can often be reimplemented. If the required parameters and a sufficiently independent reference implementation are available, the new code can be tested numerically. The same is true for many algorithms, file transformations, workflow rules, and visualizations.

But reproducing the equation does not reproduce the force field.

The parameter values encode prior fitting and scientific decisions. A neural network architecture does not recreate the learned information stored in its weights. A model implementation does not recreate the experimental datasets, benchmark history, prospective successes and failures, or accumulated knowledge that establish where the model is useful.

Implementation can often be regenerated; scientific evidence must be preserved.

This is also why provenance and licensing remain meaningful in generated software. An agent can construct original code that calls or reimplements public methods, but generation does not erase the origin of parameter sets, trained weights, algorithms, datasets, or external scientific components. Scientific software becomes easier to compose precisely because those layers can be separated more cleanly.

6 Validation becomes part of the product

The easier it becomes to generate a scientific calculation, the more important it becomes to define what kind of evidence supports it.

Molarium separates several questions that are often collapsed into the statement that a method "works."

At the lowest level, the workflow must execute and produce a technically valid artifact. At the next level, the implementation must match its declaration: equations, parameters, numerical behavior, and reference outputs should agree within defined tolerances. Retrospective performance is a separate question. Prospective transfer to new chemistry or biological situations is harder again. Ultimately, the relevant standard in drug discovery is whether a prediction improves experimental decisions.

These levels form an evidence ladder.

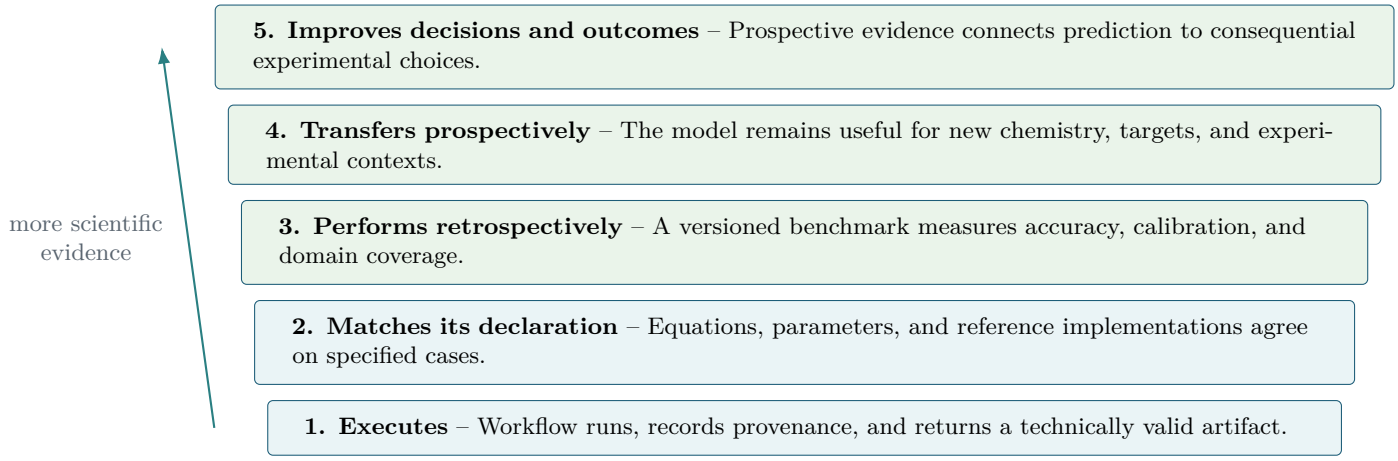

Figure 4. An evidence ladder for agentic molecular design. Automation can accelerate work at every level, but evidence from one level does not establish the next.

Molarium currently provides strongest evidence at the lower levels.

Its numerical routes are tested against independent or partially independent references using common coordinates and prepared parameter arrays. Energies and forces can be compared with OpenMM or TorchANI. Analytic forces can be challenged with finite differences. Regression fixtures test units, exceptions, constraints, deterministic replay, translation invariance, model hashes, supported domains, and failure behavior.

These comparisons matter because browser-native numerical code is not scientifically credible merely because it produces plausible-looking trajectories. A unit error can yield smooth dynamics. Two wrong implementations can agree if they share the same mistaken input. A correct numerical implementation can still instantiate an inappropriate physical model.

The hierarchy therefore prevents one type of success from being inflated into another.

This distinction is visible in Molarium's conformer work. In an early eight-molecule development panel, the homogeneous-replica workflow found a lower common Sage minimum for four molecules and added structural clusters that were absent from the initial seed lane. The median best-energy improvement was small. The result supports increased exploration and structural diversity under the defined workflow. It does not establish that the method generally predicts experimentally relevant conformational ensembles better than established alternatives.

That conclusion is scientifically less dramatic and considerably more useful.

A publication-quality claim about conformer prediction would require a chemically diverse frozen benchmark, repeated random seeds, symmetry-aware structural metrics, reference-conformer recall, matched end-to-end timing, converged optimization, uncertainty over molecules and seeds, and ideally an independent higher-level or experimental judge. The ability to specify that test is itself part of the scientific result: generation of a workflow must be accompanied by generation of the conditions under which its claims could be falsified.

Performance claims require the same discipline. WebGPU can make sufficiently large or highly batched browser workloads useful, but dispatch and synchronization overhead can make it unattractive for tiny systems. Comparison with OpenMM Reference WebAssembly does not establish superiority to optimized native CPU or CUDA software. Indeed, current comparisons with upstream native STORMM illustrate that a browser implementation may remain slower than a specialized CUDA code even while providing a large improvement over its browser baseline.

The scientifically defensible claim is therefore narrower than "GPU is faster" or "the browser replaces native

molecular simulation."

The claim is that a browser can now host sufficiently capable local molecular calculations to alter the interaction model between scientist and software. Calculations that previously required leaving the design environment can increasingly occur inside it, with the result, method identity, and provenance remaining attached to the molecular session.

7 What becomes scarce when implementation becomes cheap

Molarium turns the broader argument into a concrete stack.

The first layer is **implementation and orchestration**: interfaces, visualization, format conversion, wrappers, workflow logic, reports, workers, job control, and numerical plumbing. This layer remains important, sometimes difficult, and costly to maintain. But it is the layer whose marginal reproduction cost is falling fastest as coding agents improve.

Molarium itself is evidence of that change. A scientist did not need to wait for every useful workflow to appear on a conventional product roadmap. Missing interfaces and bounded scientific implementations could be added while the underlying scientific question was still active.

The second layer is the **scientific model**: force fields, learned potentials, protein-structure predictors, molecular representations, property models, parameters, weights, calibration, and the experimental evidence that establishes their domain of usefulness.

This layer does not become cheap merely because access becomes easier.

A docking model with an inadequate scoring function remains inadequate when invoked by an agent. A force field with a systematic error becomes more dangerous, not less, if an automated workflow generates thousands of confident predictions from it. A machine-learned property model does not acquire extrapolative validity because a language model describes its answer fluently. A protein-structure predictor does not become reliable for a particular conformational state because it is available through a better interface.

As the mechanics of invoking methods become less scarce, differences between the methods become more consequential.

The third layer is **discovery intelligence**: proprietary experimental data, negative results, project history, assay context, structural interpretation, design rationale, internal protocols, expert judgment, organizational priorities, and the remembered reasons that apparently promising ideas failed.

Molarium intentionally contains little of this third layer. It is a public workbench built primarily from public scientific components. That absence is useful because it clarifies the boundary. An organization could build project-specific interfaces around the same type of infrastructure while connecting them to internal structures, assay data, pharmacokinetics, target knowledge, medicinal-chemistry reasoning, and program objectives.

The resulting system would be more valuable not because its interface was harder to build, but because the scientific context informing its decisions was difficult to reproduce elsewhere.

This framing also changes how institutional knowledge might be operationalized.

An organization's protein-preparation procedure is more than a sequence of shell commands. It includes policies for protonation, crystallographic waters, alternate conformations, missing atoms, cofactors, unusual residues, and failure escalation. A free-energy workflow is more than launching simulations; it includes rules for perturbation design, convergence, diagnostics, structural ambiguity, and disagreement with experiment. A compound-selection process combines measured and predicted properties with uncertainty, synthetic feasibility, novelty, program objectives, and the expected information value of the next experiment.

Agentic systems make it possible to encode such processes as reusable scientific skills.

The important asset is not the prose of the prompt. It is the versioned decision process: its inputs, models, assumptions, uncertainty treatment, controls, provenance, acceptance tests, domain restrictions, escalation conditions, and accountable scientific owner.

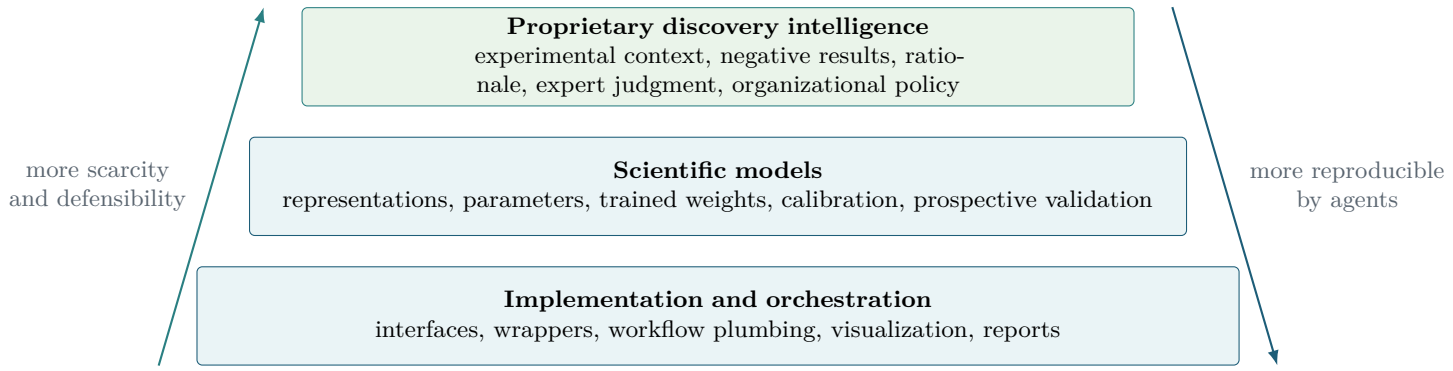

Figure 5. Three interdependent layers of value in agentic molecular design. Implementation and orchestration are becoming increasingly reproducible by coding agents. Scientific models remain differentiated by accuracy, representation, domain coverage, calibration, and validation. Proprietary discovery intelligence may be the least reproducible layer.

In this sense, the tool layer becoming cheaper does not reduce the importance of scientific software organizations. It changes where durable value resides.

8 The changing role of the scientist

There is an apparent paradox in this transition.

As software becomes easier to operate and modify, the judgment of the scientist becomes more important.

Traditional computational expertise often includes substantial operational knowledge: how to prepare a particular input, which interface setting corresponds to a numerical parameter, how to move structures between programs, how to recover a failed job, or how to reproduce a specific analysis. These skills remain useful, but agents can increasingly encode and execute them.

What agents do not remove is the need to ask whether the result makes sense.

Molarium's development repeatedly depended on such judgments. Someone had to recognize that a molecular geometry was chemically impossible. Someone had to question an energy scale rather than accepting a plausible-looking number. Someone had to distinguish a reference CPU implementation from a genuinely GPU-resident computation. Someone had to decide whether a faster benchmark was using a meaningful baseline and whether a conformer result supported exploration, prediction, or neither.

The bottleneck therefore shifts from operation toward epistemology.

What approximation is acceptable? Which result deserves an independent control? Is the reference actually independent? Does a discrepancy matter chemically? Is a model being used outside its training domain? What uncertainty should remain visible? What experiment would distinguish between competing hypotheses?

The ability to implement an answer quickly increases the value of asking the right question.

Computational scientists likewise become more rather than less important, but their highest-leverage work may shift. Less time needs to be spent constructing repetitive interfaces or manually operating routine workflows. More can be spent on physical models, representations, parameterization, sampling, validation, uncertainty, difficult edge cases, and the circumstances in which existing methods fail. Once that expertise is made explicit, parts of it can be encoded into systems that support a broader group of molecular designers.

Molarium is a small-scale example of this organizational pattern. The coding agent compressed the distance between recognizing a scientific need and changing the software. It did not remove the need to recognize the scientific need.

9 Limitations

Molarium is a case study, not a controlled experiment in software productivity or drug-discovery performance.

Its development involved one primary scientist, one evolving codebase, and a particular coding-agent workflow. There was no randomized comparison with a conventional software-development team, no blinded evaluation, and no complete reconstruction of active development time. The project therefore cannot establish a universal productivity multiplier or determine what fraction of commercial scientific software can be regenerated reliably.

Its scientific scope is also deliberately bounded.

Current browser-native molecular mechanics does not reproduce the full capability of production native simulation environments. Important features such as general periodic simulation, PME electrostatics, barostats, membranes, arbitrary OpenMM plugins, and unrestricted protein parameterization remain outside the current browser-native scope. Browser small-molecule workflows use simplified charge choices in contexts where more sophisticated charge models may be preferable. ANI-2x is restricted to its declared chemical domain. Browser OpenFold support is narrower than general protein-structure prediction workflows. Hardware and browser differences remain relevant to numerical behavior and performance.

Most importantly, implementation agreement is not molecular prediction.

A browser kernel can reproduce a declared force-field calculation while the force field itself remains inaccurate for the chemistry under study. Two implementations can agree while sharing the same problematic parameterization. A fast conformer workflow can explore more structures without finding more biologically relevant ones. A successful retrospective benchmark can fail prospectively.

These limitations are not peripheral qualifications to Molarium's argument. They are the argument.

When software is difficult to construct, implementation effort can hide the distinction between building a method and validating it. When implementation becomes easy, the remaining scientific obligations become much harder to ignore.

10 Outlook

The future suggested by Molarium is not one in which every scientist writes a private molecular-modeling package or every commercial application is replaced by generated code.

It is a future in which the interface between scientist and computation becomes considerably more fluid.

A project may construct a temporary view around a specific design question. A medicinal chemist may ask to see a series organized by permeability, clearance, structural hypothesis, and predicted affinity. A computational scientist may introduce a new diagnostic and make it available immediately. An internal model may be used alongside an open-source method and a commercial service without requiring each capability to own a separate user experience.

Under that model, the durable scientific infrastructure is not the screen.

It is the explicit molecular state, the model, its parameters and weights, the applicability domain, the validation evidence, the provenance, and the institutional knowledge governing how the result is interpreted.

Molarium represents an early version of such an environment. Its most important feature is therefore not any individual force field, neural potential, simulator, or structure predictor. It is the attempt to make diverse computational capabilities composable without allowing their scientific identities to disappear.

That requirement will become more important as generative software improves.

11 Conclusion

Molarium began as a request for a better place to inspect and manipulate molecules. It became a local-first molecular-design environment incorporating structure editing and preparation, conformer workflows, molecular

mechanics, molecular dynamics, learned potentials, protein-structure prediction, browser-native GPU computation, provenance, and reference validation.

The speed with which that environment could be expanded is technically interesting. The deeper lesson is what the process did *not* make cheap.

The coding agent could construct an interface, implement a numerical kernel, connect a workflow, and reproduce a published equation. It could not regenerate the scientific information contained in validated parameter sets, trained model weights, proprietary experimental evidence, applicability domains, or expert judgment. Nor could numerical agreement with a reference establish that the shared model was correct for the next molecule.

Molarium therefore suggests a different way to think about the coming generation of computational molecular-design systems.

Software increasingly adapts to the scientific problem rather than forcing the problem into a fixed application. The more fluid that software becomes, the more explicit its scientific contracts must become. And as the tool layer becomes easier to reproduce, the genuinely scarce assets become easier to see: better models of molecular reality, better evidence, better accumulated discovery intelligence, and scientists capable of deciding how much to trust them.

Software implementation is becoming cheap. Scientific truth is not.

AI assistance and availability

GPT-5.6 Sol was used extensively as a coding agent during the development of Molarium under human scientific direction, inspection, testing, and revision. Language-model assistance was also used in restructuring and drafting this manuscript. The authors remain responsible for the scientific claims, validation, source attribution, and final text.

Molarium source code and technical documentation are publicly available at `https://github.com/rafwiewiora/molarium`. A submitted version relying on quantitative benchmark claims should be accompanied by a frozen validation ledger recording the relevant commit, fixtures, model and asset hashes, browser and hardware environment, numerical precision, and timing boundaries.

References

[1] Bran, A. M.; Cox, S.; Schilter, O.; Baldassari, C.; White, A. D.; Schwaller, P. Augmenting Large Language Models with Chemistry Tools. *Nat. Mach. Intell.* **2024**, *6*, 525–535.

[2] Boiko, D. A.; MacKnight, R.; Kline, B.; Gomes, G. Autonomous Chemical Research with Large Language Models. *Nature* **2023**, *624*, 570–578.

[3] Haas, A.; Rossberg, A.; Schuff, D. L.; et al. Bringing the Web up to Speed with WebAssembly. *Proc. PLDI* **2017**, 185–200.

[4] W3C GPU for the Web Working Group. *WebGPU Specification*, 2025.

[5] Landrum, G. *RDKit: Open-Source Cheminformatics*, 2025 release.

[6] Riniker, S.; Landrum, G. A. Better Informed Distance Geometry: Using What We Know to Improve Conformation Generation. *J. Chem. Inf. Model.* **2015**, *55*, 2562–2574.

[7] Halgren, T. A. Merck Molecular Force Field. I. Basis, Form, Scope, Parameterization, and Performance of MMFF94. *J. Comput. Chem.* **1996**, *17*, 490–519.

[8] Rappe, A. K.; Casewit, C. J.; Colwell, K. S.; Goddard, W. A.; Skiff, W. M. UFF, a Full Periodic Table Force Field for Molecular Mechanics and Molecular Dynamics Simulations. *J. Am. Chem. Soc.* **1992**, *114*, 10024–10035.

[9] Boothroyd, S.; Madin, O. C.; Mobley, D. L.; et al. Development and Benchmarking of Open Force Field 2.0.0: The Sage Small-Molecule Force Field. *J. Chem. Theory Comput.* **2023**, *19*, 3251–3275.

[10] Eastman, P.; Galvelis, R.; Pelaez, R. P.; et al. OpenMM 8: Molecular Dynamics Simulation with Machine Learning Potentials. *J. Phys. Chem. B* **2024**, *128*, 109–116.

[11] Cerutti, D. S.; Wiewiora, R. P.; Boothroyd, S.; Sherman, W. STORMM: Structure and Topology Replica Molecular Mechanics for Chemical Simulations. *J. Chem. Phys.* **2024**, *161*, 032501.

[12] Gao, X.; Ramezanghorbani, F.; Isayev, O.; Smith, J. S.; Roitberg, A. E. TorchANI: A Free and Open Source PyTorch-Based Deep Learning Implementation of the ANI Neural Network Potentials. *J. Chem. Inf. Model.* **2020**, *60*, 3408–3415.

[13] Devereux, C.; Smith, J. S.; Huddleston, K. K.; et al. Extending the Applicability of the ANI Deep Learning Molecular Potential to Sulfur and Halogens. *J. Chem. Theory Comput.* **2020**, *16*, 4192–4202.

[14] Ahdritz, G.; Bouatta, N.; Floristean, C.; et al. OpenFold: Retraining AlphaFold2 Yields New Insights into Its Learning Mechanisms and Capacity for Generalization. *Nat. Methods* **2024**, *21*, 1514–1524.

[15] Onufriev, A.; Bashford, D.; Case, D. A. Exploring Protein Native States and Large-Scale Conformational Changes with a Modified Generalized Born Model. *Proteins* **2004**, *55*, 383–394.

[16] Schaefer, M.; Karplus, M. A Comprehensive Analytical Treatment of Continuum Electrostatics. *J. Phys. Chem.* **1996**, *100*, 1578–1599.

[17] Ryckaert, J.-P.; Ciccotti, G.; Berendsen, H. J. C. Numerical Integration of the Cartesian Equations of Motion of a System with Constraints: Molecular Dynamics of n-Alkanes. *J. Comput. Phys.* **1977**, *23*, 327–341.

[18] Andersen, H. C. RATTLE: A "Velocity" Version of the SHAKE Algorithm for Molecular Dynamics Calculations. *J. Comput. Phys.* **1983**, *52*, 24–34.

[19] Bender, A.; Cortes-Ciriano, I. Artificial Intelligence in Drug Discovery: What Is Realistic, What Are Illusions? Part 1. *Drug Discov. Today* **2021**, *26*, 511–524.

[20] Bender, A.; Cortes-Ciriano, I. Artificial Intelligence in Drug Discovery: What Is Realistic, What Are Illusions? Part 2. *Drug Discov. Today* **2021**, *26*, 1040–1052.

[21] Wigh, D. S.; Goodman, J. M.; Lapkin, A. A. A Review of Molecular Representation in the Age of Machine Learning. *WIREs Comput. Mol. Sci.* **2022**, *12*, e1603.

[22] Wilkinson, M. D.; Dumontier, M.; Aalbersberg, I. J.; et al. The FAIR Guiding Principles for Scientific Data Management and Stewardship. *Sci. Data* **2016**, *3*, 160018.

[23] Bainbridge, L. Ironies of Automation. *Automatica* **1983**, *19*, 775–779.

[24] Sculley, D.; Holt, G.; Golovin, D.; et al. Hidden Technical Debt in Machine Learning Systems. *Adv. Neural Inf. Process. Syst.* **2015**, *28*, 2503–2511.